

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO
MÔN HỌC: CÔNG NGHỆ PHẦN MỀM
NĂM HỌC 2024 – 2025**

KIỂM THỬ PHẦN MỀM

Sinh viên thực hiện: Nguyễn Thanh Tùng MSSV: 23021715

Giảng viên hướng dẫn: PGS. TS Phạm Ngọc Hùng

HÀ NỘI - 2025

1 Tổng quan về kiểm thử phần mềm

1.1 Khái niệm kiểm thử

Trước hết, ta đi vào khái niệm về thuật ngữ V&V (Kiểm chứng và thẩm định). Kiểm chứng là quá trình nhằm xác nhận rằng sản phẩm phần mềm tuân thủ đúng các yêu cầu và đặc tả kỹ thuật đã đề ra. Ngược lại, thẩm định là quá trình nhằm xác định xem sản phẩm có đáp ứng đúng nhu cầu và mong đợi của người dùng (khách hàng) hay không. Trên thực tế, kiểm chứng cần được tiến hành trước thẩm định. Việc thực hiện kiểm chứng trước có vai trò then chốt nhằm đảm bảo rằng sản phẩm đã được xây dựng đúng theo đặc tả ban đầu. Nếu bỏ qua bước này và tiến hành thẩm định trước, khi phát hiện ra lỗi, chúng ta sẽ gặp khó khăn trong việc phân biệt nguyên nhân: lỗi xuất phát từ sự sai sót trong đặc tả hay từ việc lập trình không đúng theo đặc tả. Ta có hai phương pháp để thực hiện V&V, bao gồm việc execute mã nguồn (được gọi là dynamic) và không execute mã nguồn (được gọi là static). Quá trình chạy mã nguồn để tìm ra lỗi chính là quá trình kiểm thử.

Kiểm thử là hoạt động chủ chốt nhằm đánh giá chất lượng phần mềm. Mục đích chính của kiểm thử chính là phát hiện ra lỗi phần mềm. Một kiểm thử thành công là một kiểm thử có thể phát hiện ra lỗi đó.

Trong khái niệm về kiểm thử nêu trên, có một số thuật ngữ cần làm rõ là lỗi và chất lượng phần mềm. Đầu tiên, lỗi (error) là những sơ suất xuất phát từ những sai sót do con người gây ra trong quá trình phân tích, thiết kế, lập trình hoặc các công việc liên quan. Lỗi này sẽ dẫn đến những sai sót hoặc khiếm khuyết (fault/ defect) trong sản phẩm phần mềm. Một khi lỗi sai đó được thực thi thì nó sẽ dẫn đến thất bại hoặc trục trặc (failure) - tức là hệ thống không vận hành như mong đợi. Các sự cố (incident) là những dấu hiệu hoặc triệu chứng rõ ràng của trục trặc, được người dùng hoặc hệ thống giám sát phát hiện ra. Chính vì vậy, việc kiểm thử phần mềm sẽ nhằm mục tiêu phát hiện và ngăn chặn lỗi, khiếm khuyết trước khi chúng có cơ hội gây ra trục trặc hoặc sự cố trên thực tế.

Bên cạnh đó, chất lượng phần mềm được hiểu là mức độ thỏa mãn sản phẩm so với các yêu cầu và đặc tả đã đặt ra ban đầu. Nói cách khác, chất lượng phần mềm sẽ phản ánh độ tốt của sản phẩm trong quá trình phát triển và sử dụng. Ngoài ra, để đánh giá chất lượng phần mềm, người ta có thể dựa trên nhiều tiêu chí khác nhau. Một sản phẩm chất lượng cần có tính đúng đắn, tức là nó sẽ đáp ứng chính xác các đặc tả và yêu cầu thiết kế; đồng thời, nó phải thể hiện tính hiệu quả trong hoạt động và xử lý tài nguyên. Ngoài ra, chất lượng phần mềm còn thể hiện qua các đặc tính như khả năng kiểm thử, tính dễ học và dễ sử dụng cũng như khả năng dễ bảo trì. Trong đó, độ tin cậy - một yếu tố phản ánh khả năng phần mềm vận hành ổn định, ít xảy ra lỗi trong các điều kiện sử dụng thực tế - cũng là một trong nhiều yếu tố cấu thành chất lượng, nhưng nó là chỉ số đặc biệt quan trọng trong đánh giá chất lượng phần mềm.

1.2 Vai trò của kiểm thử

Kiểm thử phần mềm đóng vai trò cực kỳ quan trọng trong quá trình phát triển phần mềm. Nó sẽ góp phần vào quá trình đảm bảo chất lượng của sản phẩm phần mềm. Đảm bảo chất lượng phần mềm bao gồm tất cả các kỹ thuật để đảm bảo chất lượng cho đầu ra của chương trình. Trong đó, kiểm thử phần mềm là một dây chuyền con trong quá trình đảm bảo chất lượng ấy. Giai đoạn kiểm thử ấy chính là một cách làm cụ thể để đảm bảo chất lượng ở pha Implementing. Thông qua việc kiểm thử phần mềm, nhà phát triển có thể đánh giá được mức độ hoàn thiện của sản phẩm, từ đó đưa ra các quyết định về việc sửa chữa, cải tiến hoặc ra mắt sản phẩm. Khi đó, kiểm thử phần mềm không chỉ là một bước cần thiết trong quy trình phát triển mà còn là một phần quan trọng trong việc đảm bảo rằng sản phẩm cuối cùng đáp ứng được các yêu cầu kỹ thuật và nhu cầu thực tế của người dùng.

Một trong những mục tiêu chính của kiểm thử phần mềm là phát hiện lỗi, sai sót trong mã nguồn, đồng thời xác minh xem sản phẩm có hoạt động đúng như mong đợi hay không. Quy trình này không chỉ giới hạn ở việc tìm kiếm lỗi mà còn liên quan đến việc đánh giá khả năng hoạt động của phần mềm trong các điều kiện khác nhau. Kiểm thử giúp xác định các vấn đề tiềm ẩn có thể ảnh hưởng đến hiệu suất hoặc bảo mật của sản phẩm. Điều này giúp giảm thiểu khả năng phát sinh các lỗi sau khi sản phẩm được đưa vào sử dụng, điều mà có thể gây ảnh hưởng xấu đến uy tín của công ty, sự hài lòng của người dùng và ngay cả chi phí bảo trì sau này.

Một yếu tố quan trọng khác là kiểm thử phần mềm giúp nhà phát triển giảm thiểu rủi ro. Rủi ro là yếu tố không thể tránh khỏi trong bất kỳ dự án phần mềm nào, nhưng việc kiểm thử thường xuyên giúp phát hiện các nguy cơ tiềm ẩn, từ đó có biện pháp xử lý kịp thời. Kiểm thử cũng giúp đội ngũ phát triển có cái nhìn tổng quan về chất lượng sản phẩm, đồng thời giúp các nhà quản lý dự đoán và lên kế hoạch về thời gian, chi phí và nguồn lực cần thiết để hoàn thiện sản phẩm.

1.3 Kiểm thử tĩnh và kiểm thử động

Trong thực tế, quy trình kiểm thử phần mềm có hai loại chính: kiểm thử tĩnh và kiểm thử động. Kiểm thử tĩnh liên quan đến việc phân tích mã nguồn và tài liệu phần mềm mà không cần thực thi phần mềm. Đây là một phương pháp để đánh giá các yếu tố như cấu trúc mã, khả năng bảo trì, sự tuân thủ các quy chuẩn lập trình, và tính dễ hiểu của mã. Kiểm thử tĩnh giúp phát hiện lỗi sớm trong quá trình phát triển mà không cần chạy phần mềm, giúp tiết kiệm thời gian và tài nguyên.

Trong khi đó, kiểm thử động là quá trình thực thi phần mềm để kiểm tra chức năng và hiệu suất của nó trong các tình huống thực tế. Kiểm thử động bao gồm các phương pháp như kiểm thử đơn vị, kiểm thử hệ thống, kiểm thử tích hợp và kiểm thử chấp nhận. Đây là các bước quan trọng để đảm bảo phần mềm hoạt động như dự kiến trong môi trường thực tế và đáp ứng được các yêu cầu của người dùng cuối. Kiểm thử động có thể giúp xác định các vấn đề mà kiểm thử tĩnh không thể phát hiện được, như các lỗi liên quan đến hiệu suất hoặc các vấn đề tương tác giữa các phần mềm hoặc hệ thống.

1.4 Ca kiểm thử

Có thể nói, cốt lõi của kiểm thử phần mềm dựa trên phân tích động là việc xác định tập các ca kiểm thử sao cho chúng có khả năng phát hiện nhiều nhất các lỗi của hệ thống cần kiểm thử. Như vậy, một ca kiểm thử được thiết kế sẽ nhằm mục đích đó. Cấu trúc của một ca kiểm thử bao gồm:

Định danh (tên) của ca kiểm thử
Mục đích
Tiền điều kiện
Đầu vào
Đầu ra mong đợi
Hậu điều kiện
Lịch sử thực hiện ca kiểm thử:
Ngày Kết quả thực tế Phiên bản Kiểm thử viên

Hình 1: Thông tin của một ca kiểm thử

Hình trên mô tả những thông tin cần có của một ca kiểm thử. Trong đó, các ca kiểm thử cần phải được định danh bằng tên/ chỉ số và lý do của nó tồn tại (hay còn gọi là mục đích của ca kiểm thử). Tiếp theo, kiểm thử viên cần định nghĩa tiền điều kiện và đầu vào của ca kiểm thử. Tiền điều kiện (pre-condition) tức là điều kiện cần thỏa mãn trước khi tiến hành một ca kiểm thử. Dữ liệu đầu vào sẽ thực sự được xác định bởi phương pháp kiểm thử. Thông tin tiếp theo cần đưa vào là đầu ra mong đợi và hậu điều kiện. Ngoài ra, kiểm thử viên cũng nên bổ sung thêm lịch sử tiến hành của một ca kiểm thử, bao gồm việc chúng được chạy bởi ai và chạy khi nào, kết quả mỗi lần chạy ra sao và được chạy trên phiên bản nào của phần mềm.

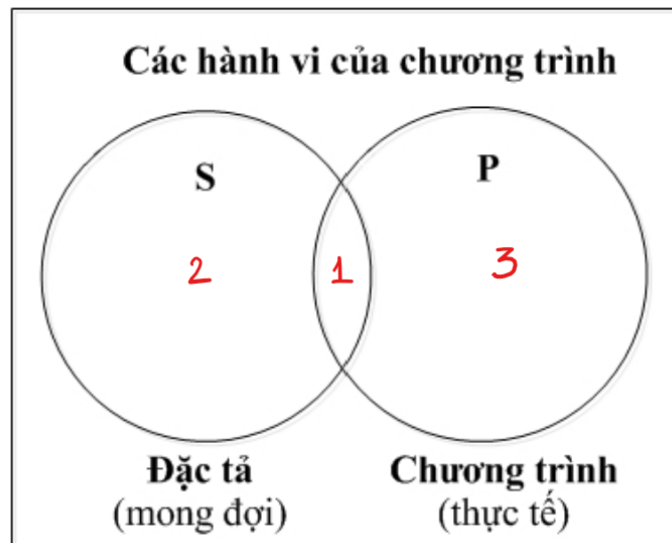
1.5 Quá trình kiểm thử

Quá trình kiểm thử phần mềm là một quá trình lặp gồm nhiều bước. Đầu tiên, ta có một hệ thống cần kiểm thử (system under test). Nó có thể là một đơn vị (unit), một thành phần (component), một hệ thống (system),... Nó bao gồm nhiều mức độ khác nhau tùy thuộc vào mức độ kiểm thử. Tiếp theo ta có một bộ ca kiểm thử bao gồm định danh của ca kiểm thử, mục đích, tiền điều kiện, đầu vào, đầu ra mong đợi, hậu điều kiện. Khi đó, kiểm thử viên sẽ sử dụng bộ ca kiểm thử đó để kiểm thử cho hệ thống cần kiểm thử đó. Sau khi thực hiện kiểm thử xong, kiểm thử viên sẽ xuất ra một báo cáo kiểm thử (testing report), bao gồm thêm các thông số như lịch sử thực hiện ca kiểm thử, kết quả kiểm thử. Khi có báo cáo kiểm thử thì sẽ có hai trường hợp xảy ra. Nếu như hệ thống đó vượt qua tất cả các bộ ca kiểm thử, thì ta không cần làm gì thêm. Tuy nhiên, nếu như tồn tại một ca kiểm thử thực hiện không thành công, thì ta cần phải xác định lại xem lỗi của nó nằm ở đâu thông qua quá trình debugging. Có thể nói, nếu kiểm thử (testing) là quá trình phát hiện lỗi thì debugging sẽ là quá trình tìm kiếm xem lỗi nằm ở chỗ nào. Sau đó, ta sẽ thực hiện chỉnh sửa để cho ra phiên bản thứ hai của system under test. Sau đó, ta lặp lại quá trình kiểm thử như vậy đối với phiên bản thứ hai. Ta sẽ thực hiện nhiều lần như thế cho đến khi hệ thống không còn lỗi.

2 Kiểm thử hộp trắng và kiểm thử hộp đen

2.1 Thế nào là một ca kiểm thử tốt?

Ta cùng xem xét hình dưới đây.



Hình 2: Các hành vi của một chương trình

Ta có hai vòng tròn lớn tương đương với đặc tả mong đợi và chương trình thực tế. Trong đó, vùng số 1 là vùng có đặc tả và được Implementing, vùng số 2 là vùng có đặc tả nhưng không được Implementing, vùng số 3 là vùng không có trong đặc tả nhưng được Implementing. Một bộ ca kiểm thử tốt là một bộ ca kiểm thử có thể bao phủ hết cả ba vùng này. Chính vì vậy, một số phương pháp kiểm thử ra đời để thực hiện kiểm thử sao cho bộ ca kiểm thử sinh ra là tốt nhất, bao gồm kiểm thử hộp trắng và kiểm thử hộp đen.

2.2 Kiểm thử hộp trắng

Kiểm thử hộp trắng (hay còn gọi là kiểm thử cấu trúc) là một kỹ thuật kiểm thử phần mềm mà trong đó người kiểm thử cần có hiểu biết chi tiết về cấu trúc nội bộ và logic xử lý của chương trình. Kiểm thử hộp trắng sẽ cho phép khai thác trực tiếp mã nguồn hoặc mô hình thiết kế để xác định các trường hợp kiểm thử phù hợp. Ngoài ra, các ca kiểm thử trong kiểm thử hộp trắng được thiết kế dựa trên các đường đi có thể xảy ra trong chương trình. Việc xác định các đường đi, đồ thị cho phép xác định các chỉ số của độ phủ kiểm thử như độ phủ dòng lệnh (statement coverage), độ phủ nhánh (branch coverage), độ phủ đường (path coverage),... Thông qua các độ đo về độ bao phủ, nhóm phát triển và kiểm thử có thể kiểm soát tiến độ, phát hiện những phần mã chưa được kiểm tra đầy đủ, từ đó cải thiện chất lượng phần mềm một cách hệ thống.

Khi biểu diễn dưới dạng biểu đồ, kiểm thử hộp trắng sẽ cho phép triển khai các ca kiểm thử thuộc vùng số 1 và vùng số 3. Tức là các ca kiểm thử sẽ được sinh ra từ mã nguồn để thực hiện kiểm thử. Khi đó, bộ ca kiểm thử sẽ thực hiện kiểm thử được những trường hợp có đặc tả và được Implementing và không có đặc tả nhưng vẫn được Implementing.

Ưu điểm của kiểm thử hộp trắng là nó cho phép kiểm tra chi tiết logic bên trong của chương trình, từ đó giúp phát hiện sớm các lỗi tiềm ẩn như điều kiện sai, vòng lặp vô hạn, hoặc đoạn mã không được thực thi. Do người kiểm thử có kiến thức về cấu trúc mã nguồn, họ có thể thiết kế các ca kiểm thử để đảm bảo bao phủ mã tốt hơn, giúp tăng độ tin cậy và chất lượng phần mềm. Điều này đặc biệt hữu ích trong việc tối ưu hóa mã và xác minh rằng các dòng lệnh thực sự thực hiện đúng chức năng mong muốn.

Nhược điểm của kiểm thử hộp trắng chính là việc nó yêu cầu một chi phí cao, thời gian thực hiện lớn cũng như yêu cầu người kiểm thử phải hiểu rõ về mã nguồn, cấu trúc bên trong của hệ thống. Điều này sẽ làm cho việc kiểm thử trở nên phức tạp hơn rất nhiều. Ngoài ra, kiểm thử hộp trắng chỉ quan tâm đến cấu trúc bên trong của mã nguồn

mà không quan tâm đến đặc tả bên ngoài thế nào nên quá trình kiểm thử này sẽ làm bỏ qua những đặc tả cần thiết cho nó.

2.3 Kiểm thử hộp đen

Kiểm thử hộp đen (hay còn gọi là kiểm thử chức năng) là một phương pháp kiểm thử phần mềm trong đó người kiểm thử tập trung vào chức năng và hành vi bên ngoài của hệ thống mà không cần quan tâm đến cấu trúc nội bộ hay cách thức cài đặt của chương trình. Khái niệm kiểm thử hộp đen xuất phát từ quan điểm coi chương trình phần mềm như một hàm ánh xạ từ miền dữ liệu đầu vào tới miền dữ liệu đầu ra. Trong thực hành kiểm thử phần mềm, kiểm thử hộp đen thường sử dụng các kỹ thuật như phân hoạch tương đương, phân tích giá trị biên, bảng quyết định, biểu đồ trạng thái, và kiểm thử luồng dữ liệu nhằm đảm bảo rằng mọi hành vi chức năng của hệ thống đều được kiểm tra đầy đủ. Mục tiêu chính là phát hiện các lỗi liên quan đến chức năng bị thiếu, không đúng, hoặc không phù hợp với yêu cầu người dùng.

Khi biểu diễn dưới dạng biểu đồ, kiểm thử hộp đen sẽ cho phép khai thác các ca kiểm thử thuộc vùng số 1 và vùng số 2. Tức là các ca kiểm thử sẽ được sinh ra từ đặc tả để thực hiện kiểm thử. Khi đó, bộ ca kiểm thử sẽ thực hiện kiểm thử được những trường hợp có đặc tả và được Implementing và có đặc tả nhưng không được Implementing.

Ưu điểm của kiểm thử hộp đen tập trung vào đầu vào và đầu ra của phần mềm mà không cần quan tâm đến cấu trúc bên trong, giúp đánh giá phần mềm từ góc nhìn của đặc tả yêu cầu. Phương pháp này phù hợp với việc kiểm tra chức năng, đảm bảo rằng hệ thống đáp ứng đúng yêu cầu đã đặt ra. Ngoài ra, kiểm thử hộp đen dễ tiếp cận hơn với người không phải lập trình viên và có thể áp dụng được ngay cả khi mã nguồn không sẵn có.

Tuy nhiên, nhược điểm của kiểm thử hộp đen là nó không kiểm tra được cấu trúc bên trong nên có thể bỏ sót các lỗi ẩn trong logic xử lý nội tại hoặc mã không được thực thi. Việc thiết kế ca kiểm thử có thể không đảm bảo bao phủ hết tất cả các nhánh, điều kiện hoặc tình huống biên. Bên cạnh đó, nếu tài liệu yêu cầu không đầy đủ hoặc mơ hồ, việc xác định đầu ra mong đợi có thể gây khó khăn và làm giảm hiệu quả kiểm thử.

2.4 Kiểm thử hộp đen và kiểm thử hộp trắng có thay thế được cho nhau không?

Kiểm thử hộp đen và kiểm thử hộp trắng là hai phương pháp kiểm thử phần mềm với cách tiếp cận khác nhau, mỗi phương pháp tập trung vào những khía cạnh riêng biệt của hệ thống. Do đó, chúng không thể thay thế hoàn toàn cho nhau, mà thực tế cần phải kết hợp để đảm bảo quá trình kiểm thử đạt hiệu quả toàn diện.

Một bộ ca kiểm thử tốt phải đảm bảo đi qua hết cả 3 vùng, bao gồm vùng số 1 là vùng có đặc tả và được Implementing, vùng số 2 là vùng có đặc tả nhưng không được Implementing, vùng số 3 là vùng không có trong đặc tả nhưng được Implementing. Kiểm thử hộp trắng có thể đảm bảo được vùng số 1 và vùng số 3. Kiểm thử hộp đen có thể đảm bảo được vùng số 1 và vùng số 2. Chính vì vậy, để đạt được hiệu quả tốt nhất (tức là cả 3 vùng đều được đảm bảo kiểm thử) thì ta nên kết hợp hai phương pháp kiểm thử hộp đen và kiểm thử hộp trắng.

Kiểm thử hộp trắng tập trung vào việc kiểm tra cấu trúc bên trong của phần mềm. Người kiểm thử cần hiểu rõ mã nguồn, thuật toán, luồng điều khiển và dữ liệu bên trong hệ thống để thiết kế các ca kiểm thử nhằm đảm bảo các đường đi logic đều được thực thi. Tuy nhiên, một điểm hạn chế lớn của kiểm thử hộp trắng là số lượng đường đi logic có thể là vô hạn trong những hệ thống phức tạp, khiến việc kiểm thử trở nên không khả thi về mặt chi phí và thời gian nếu cố gắng bao phủ toàn bộ. Ngoài ra, kiểm thử hộp trắng chủ yếu kiểm tra những gì đã được lập trình, chứ không kiểm tra được liệu phần mềm đã thực hiện đúng yêu cầu người dùng hay chưa. Hơn nữa, kiểm thử hộp trắng không thể phát hiện những ca kiểm thử còn thiếu, tức là không đảm bảo việc kiểm tra toàn diện các tình huống sử dụng thực tế.

Ngược lại, kiểm thử hộp đen coi phần mềm như một hệ thống đóng, trong đó người kiểm thử chỉ cần quan tâm tới mối quan hệ giữa đầu vào và đầu ra mà không cần biết về cấu trúc nội bộ. Cách tiếp cận này thuận lợi trong việc kiểm tra chức năng của phần mềm so với đặc tả yêu cầu. Tuy nhiên, kiểm thử hộp đen cũng tồn tại những thách thức đáng kể. Do không dựa trên mã nguồn, việc thiết kế ca kiểm thử có thể dễ dàng dẫn đến sự bùng nổ tổ hợp

về số lượng ca kiểm thử, đặc biệt khi xét tới tất cả khả năng đầu vào hợp lệ và không hợp lệ. Bên cạnh đó, trong kiểm thử hộp đen, người kiểm thử thường không chắc chắn liệu một ca kiểm thử cụ thể có khả năng phát hiện ra lỗi cụ thể nào hay không, vì không có sự tương quan trực tiếp giữa ca kiểm thử và các phần tử bên trong mã nguồn. Tuy vậy, kiểm thử hộp đen lại rất thích hợp cho kiểm thử hệ thống và kiểm thử chấp nhận, nơi mục tiêu là đảm bảo phần mềm đáp ứng các yêu cầu chức năng và nghiệp vụ thực tế.

Chính vì những hạn chế nội tại của từng phương pháp, việc chỉ sử dụng duy nhất một trong hai kỹ thuật sẽ không đảm bảo kiểm thử được toàn diện. Thực tế, kiểm thử hộp trắng và kiểm thử hộp đen là hai thái cực trong kiểm thử phần mềm, mỗi kỹ thuật giải quyết một mặt của vấn đề chất lượng sản phẩm. Kiểm thử hộp trắng chủ yếu kiểm tra tính đúng đắn nội bộ, còn kiểm thử hộp đen kiểm tra tính đầy đủ và đúng đắn chức năng bên ngoài. Chính vì thế, việc kết hợp cả hai phương pháp kiểm thử sẽ đem lại hiệu quả cao nhất cho lập trình viên.

3 Bài toán sử dụng phương pháp kiểm thử phân hoạch tương đương và giá trị biên

3.1 Bài toán

Đặc tả bài toán: Xét tuyển Đại học. Một trường Đại học ở Mỹ sau khi nhận được hồ sơ của các ứng viên trên toàn thế giới, sau một thời gian tiến hành phân tích, lọc hồ sơ đã đưa ra những tiêu chí sau để nhận sinh viên:

- Chứng minh tài chính
- Điểm SAT
- Điểm bài luận của sinh viên
- Số thư giới thiệu của sinh viên

Các trạng thái của việc xét tuyển hồ sơ của sinh viên bao gồm:

- Accepted: Chấp nhận nhập học
- Waitlisted: Đưa sinh viên vào danh sách chờ, nếu sinh viên Accepted không xác nhận nhập học thì sẽ xét đến danh sách chờ.
- Rejected: Từ chối nhập học.

Cụ thể như sau:

Input:

- Tài chính: (tên biến: finance): 1 số nguyên với $finance \geq 0$.
- Điểm SAT: (tên biến: sat): 1 số nguyên với $0 \leq sat \leq 1600$
- Điểm bài luận: (tên biến: essay): 1 số nguyên với $0 \leq essay \leq 100$.
- Số thư giới thiệu của sinh viên: (tên biến: letter): 1 số nguyên với $0 \leq letter$

Output:

- 1: Accept: Chấp nhận nhập học
- 2: Waitlisted: Đưa vào danh sách chờ
- 3: Rejected: Từ chối nhập học

Luồng xử lý:

- Nếu $finance < 50000$ thì Rejected.
- Nếu $sat < 1300$ thì Rejected.
- Nếu $letter \leq 1$ thì Rejected.
- Nếu $essay < 70$ thì Rejected.
- Nếu $finance \geq 50000$ và $sat \geq 1300$ và $letter > 1$ và $essay \geq 70$ thì Waitlisted.
- Nếu $finance \geq 50000$ và $sat \geq 1450$ và $letter > 3$ và $essay \geq 85$ thì Accepted
- Nếu input không thỏa mãn: Error.

Điều kiện đầu vào	Điều kiện đầu ra
$finance < 0$ or $(sat < 0$ or $sat > 1600)$ or $(essay < 0$ or $essay > 100)$ or $letter < 0$	Error
$finance < 50000$ or $sat < 1300$ or $letter \leq 1$ or $essay < 70$	Rejected
$finance \geq 50000$ and $sat \geq 1300$ and $letter > 1$ and $essay \geq 70$	Waitlisted
$finance \geq 50000$ and $sat \geq 1450$ and $letter > 3$ and $essay \geq 85$	Accepted

Bảng 1: Điều kiện đầu vào và đầu ra

3.2 Phân tích giá trị biên

Phương pháp giá trị biên nói rằng mỗi ca kiểm thử cần lấy các giá trị ngay sát ranh giới của các khoảng điểm, bởi vì lỗi thường xảy ra ở các điểm chuyển tiếp này.

- Biên và cận biên của biến finance: -1, 0, 1, 49999, 50000, 50001
- Biên và cận biên của biến sat: -1, 0, 1, 1299, 1300, 1301, 1449, 1450, 1451, 1599, 1600, 1601
- Biên và cận biên của biến essay: -1, 0, 1, 69, 70, 71, 84, 85, 86, 99, 100, 101

- Biên và cận biên của biến letter: -1, 0, 1, 2, 3, 4.

Với mỗi biên/ cận biên của từng biến, ta sẽ lấy nom của các biến còn lại để tạo thành 1 ca kiểm thử. Từ đó ta có bảng bộ ca kiểm thử như sau:

Test case	finance	sat	essay	letter	Kết quả mong đợi
TC1	-1	1500	90	5	Error
TC2	0	1500	90	5	Rejected
TC3	1	1500	90	5	Rejected
TC4	49999	1500	90	5	Rejected
TC5	50000	1500	90	5	Accepted
TC6	50001	1500	90	5	Accepted
TC7	60000	-1	90	5	Error
TC8	60000	0	90	5	Rejected
TC9	60000	1	90	5	Rejected
TC10	60000	1299	90	5	Rejected
TC11	60000	1300	90	5	Waitlisted
TC12	60000	1301	90	5	Waitlisted
TC13	60000	1449	90	5	Waitlisted
TC14	60000	1450	90	5	Accepted
TC15	60000	1451	90	5	Accepted
TC16	60000	1599	90	5	Accepted
TC17	60000	1600	90	5	Accepted
TC18	60000	1601	90	5	Error
TC19	60000	1500	-1	5	Error
TC20	60000	1500	0	5	Rejected
TC21	60000	1500	1	5	Rejected
TC22	60000	1500	69	5	Rejected
TC23	60000	1500	70	5	Waitlisted
TC24	60000	1500	71	5	Waitlisted
TC25	60000	1500	84	5	Waitlisted
TC26	60000	1500	85	5	Accepted
TC27	60000	1500	86	5	Accepted
TC28	60000	1500	99	5	Accepted
TC29	60000	1500	100	5	Accepted
TC30	60000	1500	101	5	Error
TC31	60000	1500	90	-1	Error
TC32	60000	1500	90	0	Rejected
TC33	60000	1500	90	1	Rejected
TC34	60000	1500	90	2	Waitlisted
TC35	60000	1500	90	3	Waitlisted
TC36	60000	1500	90	4	Accepted

Bảng 2: Bảng test case theo phương pháp giá trị biên

3.3 Phân hoạch tương đương

Phương pháp phân hoạch tương đương giúp chia đều các giá trị đầu vào thành các tập con, sao cho tất cả các giá trị trong cùng 1 tập con được xử lý giống nhau, tức là chúng cho ra một kết quả tương tự.

Ở đây, với tập đầu vào gồm 3 đầu vào là số tín chỉ, năm sinh viên học, điểm GPA kì trước, ta chia thành các vùng tương đương sau:

- Những vùng tương đương không hợp lệ:

$$D_1 = \{\text{finance, sat, essay, letter} \mid \text{finance} < 0\}$$

$$D_2 = \{\text{finance, sat, essay, letter} \mid \text{sat} < 0\}$$

$$D_3 = \{\text{finance, sat, essay, letter} \mid \text{sat} > 1600\}$$

$$D_4 = \{\text{finance, sat, essay, letter} \mid \text{essay} < 0\}$$

$$D_5 = \{\text{finance, sat, essay, letter} \mid \text{essay} > 100\}$$

$$D_6 = \{\text{finance, sat, essay, letter} \mid \text{letter} < 0\}$$

- Những vùng tương đương hợp lệ:

$$D_7 = \{\text{finance, sat, essay, letter} \mid 0 \leq \text{finance} < 50000\}$$

$$D_8 = \{\text{finance, sat, essay, letter} \mid 0 \leq \text{sat} < 1300\}$$

$$D_9 = \{\text{finance, sat, essay, letter} \mid 0 \leq \text{essay} < 70\}$$

$$D_{10} = \{\text{finance, sat, essay, letter} \mid 0 \leq \text{letter} < 2\}$$

$$D_{11} = \{\text{finance, sat, essay, letter} \mid \text{finance} > 50000, \text{essay} > 70, \text{letter} > 1, 1300 \leq \text{sat} < 1450\}$$

$$D_{12} = \{\text{finance, sat, essay, letter} \mid \text{finance} > 50000, 70 \leq \text{essay} < 85, \text{letter} > 1, \text{sat} \geq 1300\}$$

$$D_{13} = \{\text{finance, sat, essay, letter} \mid \text{finance} > 50000, \text{essay} \geq 70, 1 \leq \text{letter} \leq 3, \text{sat} \geq 1300\}$$

$$D_{14} = \{\text{finance, sat, essay, letter} \mid \text{finance} > 50000, \text{essay} \geq 85, \text{letter} \geq 4, \text{sat} \geq 1450\}$$

Mỗi các vùng tương đương nhỏ hơn được chia ra từ vùng tương đương lớn hơn sẽ bao hàm cả tính chất của vùng tương đương lớn đó.

Như vậy, để đảm bảo bao phủ đầy đủ các vùng tương đương, ta cần phải kiểm thử các vùng từ 1 đến 13

Test case	finance	sat	essay	letter	Kết quả mong đợi	Lớp tương đương
TC1	-200	1500	90	5	Error	D_1
TC2	60000	-20	90	5	Error	D_2
TC3	60000	1700	90	5	Error	D_3
TC4	60000	1500	-20	5	Error	D_4
TC5	60000	1500	120	5	Error	D_5
TC6	60000	1500	90	-2	Error	D_6
TC7	30000	1500	90	5	Rejected	D_7
TC8	60000	1000	90	5	Rejected	D_8
TC9	60000	1400	90	5	Waitlisted	D_{11}
TC10	60000	1500	40	5	Rejected	D_9
TC11	60000	1500	80	5	Waitlisted	D_{12}
TC12	60000	1500	90	1	Rejected	D_{10}
TC13	60000	1500	90	2	Waitlisted	D_{13}
TC14	60000	1500	90	5	Accepted	D_{14}

Bảng 3: Bảng test case theo phương pháp phân hoạch tương đương

3.4 Bảng quyết định

Ta thực hiện lập bảng quyết định đối với từng đầu vào và đầu ra như sau:

Điều kiện	1	2	3	4	5	6	7	8
c1: finance $\geq$ 50000? and 0 $\leq$ sat $\leq$ 1600? and 0 $\leq$ essay $\leq$ 100? and 0 $\leq$ letter?	F	T	T	T	T	T	T	T
c2: 0 $\leq$ sat $<$ 1300?	-	T	F	F	F	F	F	F
c3: 0 $\leq$ essay $<$ 69?	-	-	T	F	F	F	F	F
c4: 0 $\leq$ letter $\leq$ 1?	-	-	-	T	F	F	F	F
c5: 1300 $\leq$ sat $<$ 1450?	-	-	-	-	T	F	F	F
c6: 70 $\leq$ essay $<$ 85?	-	-	-	-	-	T	F	F
c7: 2 $\leq$ letter $\leq$ 3?	-	-	-	-	-	-	T	F
Số luật	64	32	16	8	4	2	1	1
Hành động								
a1: Error	X							
a2: Rejected		X	X	X				
a3: Waitlisted					X	X	X	
a4: Accepted								X

Bảng 4: Bảng quyết định

Khi đó, bộ ca kiểm thử sẽ như sau:

Test case	finance	sat	essay	letter	Kết quả mong đợi
TC1	-200	-200	-20	-5	Error
TC2	60000	1000	90	5	Rejected
TC3	60000	1500	40	5	Rejected
TC4	60000	1500	90	1	Rejected
TC5	60000	1400	90	5	Waitlisted
TC6	60000	1500	80	5	Waitlisted
TC7	60000	1500	90	2	Waitlisted
TC8	60000	1500	90	5	Accepted

Bảng 5: Bảng test case theo phương pháp bảng quyết định